

Architecting Navigation for Real-World Flows

Sukritha KK

Product Engineer – iOS

Strollby, UST



The Evolution of Navigation

SwiftUI's navigation capabilities have significantly advanced, offering developers more robust and flexible ways to manage user flows.

Early SwiftUI

Initial versions of SwiftUI introduced `NavigationView`, which provided basic hierarchical navigation.

iOS 16+ Standard

`NavigationView` was deprecated for iOS 16 and later, making `NavigationStack` the recommended and modern solution for complex navigation.

Introduction of `NavigationStack`

Apple unveiled `NavigationStack`, bringing a more powerful, state-driven approach akin to UIKit's `UINavigationController`.

Why Navigation Needed Rethinking

Before `NavigationStack` became the standard, SwiftUI navigation often presented significant challenges, especially for complex real-world applications.

Coupled View Tree

Navigation was tightly bound to the view hierarchy via `NavigationLink`, making programmatic, state-driven navigation and scalable architectures harder to achieve. Destination views were often declared directly, limiting flexibility.

Unpredictable Stack Pops

Updates to bound data deep within the view hierarchy could cause the navigation stack to unexpectedly pop back to previous screens.

Messy Asynchronous Flows

Coordinating navigation with asynchronous operations, like fetching data before transitioning, often led to complex and difficult-to-manage logic.

Sheet & Full Screen Cover Issues

Implementing nested navigation paths inside sheets and full screen covers frequently resulted in broken path handling and inconsistent behaviour, complicating modal presentations.



Our Solution: Centralized, Type-Safe Routing

To overcome the limitations of SwiftUI's earlier navigation patterns, we developed a robust solution built on several core principles:



Explicit, Centralised Routing

We established a clear, single point of control for all routing decisions, simplifying debugging.



Type-Safe Route Definitions

By defining routes as enums, we ensure compile-time safety, preventing accidental navigation to invalid states.



Single Source of Truth

A singleton router holds the `NavigationPath`, acting as the definitive source for the app's current navigation state, making it predictable and manageable.



Async-Friendly by Design

Decoupling the router from the UI allows for seamless integration with asynchronous operations, ensuring smooth transitions upon completion of network requests or other background tasks.

The Three-Stack Architecture

Our "Three-Stack Architecture" is designed to manage complex real-world navigation flows within SwiftUI by logically separating distinct navigation contexts. This approach ensures each part of the app maintains its own navigation state, leading to a more robust and predictable user experience.

1

Primary Stack

The **Primary stack** handles the main application features, such as hotel bookings, flight searches, and user profiles. This is where the core user journey unfolds, managed by its own dedicated router.

- Hotel booking flows
- Flight reservation process
- User profile management

1

Secondary Stack

The **Secondary stack** is dedicated to secondary flows, often presented within a sheet. This stack manages its own navigation path without interfering with the primary user experience.

- In-app support chats
- Review submission forms
- Dynamic content displays

1

Tertiary Stack

The **Tertiary stack** is reserved for transactional, flows like authentication and payment processes. These are typically presented in full-screen covers.

- Login, OTP, and signup sequences
- Secure payment workflows
- Account verification steps

Router Foundation

Our centralised, type-safe routing system is built upon a foundational `NavigationRouter` class, which manages the application's navigation state. This class leverages SwiftUI's `NavigationPath` to provide predictable and controllable navigation within each distinct stack.

```
@MainActor
class NavigationRouter: ObservableObject {
    @Published var path = NavigationPath()

    func dismiss(_ last: Int = 1) {
        guard path.count > last else { return }
        path.removeLast(last)
    }

    func dismissAll() {
        path = NavigationPath()
    }
}
```

Singleton Router Pattern

Building on our `NavigationRouter` foundation, we implement specialised router classes for each of our "Three Stacks" (Primary, Secondary, Tertiary). Each of these is designed as a singleton, ensuring a single, dedicated point of control for its respective navigation flow.

```
@MainActor
class PrimaryNavigation: NavigationRouter {
    static let router = PrimaryNavigation()

    override private init() {}

    func present(_ screen: Path) {
        path.append(screen)
    }
}

class SecondaryNavigation: NavigationRouter {
    static let router = SecondaryNavigation()

    override private init() {}

    func present(_ screen: Path) {
        path.append(screen)
    }
}

class TertiaryNavigation: NavigationRouter {
    static let router = TertiaryNavigation()

    override private init() {}

    func present(_ screen: Path) {
        path.append(screen)
    }
}
```

Defining Type-Safe Routes

Our navigation system relies on defining all possible navigation paths using `enums`, which are inherently `Hashable`. This approach brings strong type-safety and clarity to the routing logic.

```
extension PrimaryNavigation {
    enum Path: Hashable {
        case profile(ProfilePath)
        case hotel(HotelPath)
        case flight(FlightPath)
        case wishList(WishListPath)
    }

    enum WishListPath: Hashable { case wishlistItems(WishListModel) }

    enum ProfilePath: Hashable { case settings; case regionAndCurrency }

    enum HotelPath: Hashable {
        case landing
        case listing(HotelInputModel)
        case details(HotelInputModel)
        case imageGallery(String)
    }

    enum FlightPath: Hashable {
        case landing
        case listing(FlightInputModel)
        case details(String?)
        case review(String)
        case order(String)
        case cancel(FlightOrderModel)
    }
}
```

For the Primary stack, routes are defined with nested enums, allowing for deeply structured and context-specific navigation. For instance, a `.profile(ProfilePath)` case directs to a profile-related flow, managed by its own sub-enum.

```
extension TertiaryNavigation {
    enum Path: Hashable {
        case main
        case login(LoginPath)
    }

    enum LoginPath: Hashable {
        case emailSignIn
        case phoneOTPVerify(PhoneAuthViewModel)
        case emailOTPVerify(EmailSignUpViewModel)
        case emailSignUp(String)
        case phoneSignUp
        case emailPasswordLogin(String, AuthUserModel)
    }
}
```

Mapping Routes to Screens

With our type-safe routes defined, the next crucial step is to map each `Path` case to its corresponding SwiftUI `View`. This is achieved using SwiftUI's `navigationDestination(for:action:)` modifier, ensuring that the correct screen is presented when a specific navigation path is encountered.

```
extension View {
    public func primaryNavigationDestinations() -> some View {
        navigationDestination(for: PrimaryNavigation.Path.self) { path in
            switch path {
            case let .profile(screen):
                switch screen {
                case .settings:
                    ProfileSettingsScreen()
                case .regionAndCurrency:
                    RegionAndCurrencySelectScreen()
                }
            case let .hotel(screen):
                switch screen {
                case .landing:
                    HotelLandingScreen()
                case let .listing(input):
                    HotelListingScreen(input)
                case let .details(input):
                    HotelDetailsScreen(input)
                case let .imageGallery(input):
                    HotelPhotoGalleryScreen(hotelId: input)
                }

            case let .flight(screen):
                switch screen {
                case .landing:
                    FlightLandingScreen()
                case let .listing(input):
                    FlightListingScreen(input)
                case let .details(id):
                    FlightDetailsScreen(id)
                case let .review(id):
                    FlightReviewScreen(id)
                case let .order(id):
                    FlightOrderScreen(id)
                case let .cancel(flightOrder):
                    FlightCancellationReviewScreen(order: flightOrder)
                }
            case let .wishList(screen):
                switch screen {
                case let .wishlistItems(input):
                    WishListItemsScreen(input)
                }
            }
        }
    }
}
```

Each navigation stack (Primary, Secondary, Tertiary) gets its own extension on `View`, where the `navigationDestination` modifier is applied. Inside, a `switch` statement handles each `Path` case, instantiating and presenting the appropriate SwiftUI screen, often passing along necessary data.

```
extension View {
    public func tertiaryNavigationDestinations() -> some View {
        navigationDestination(for: TertiaryNavigation.Path.self) { path in
            switch path {
            case .main:
                EmptyView()
            case let .login(screen):
                switch screen {
                case .emailSignIn:
                    EmailAuthScreen()
                case let .phoneOTPVerify(viewModel):
                    OTPVerificationScreen(viewModel: viewModel)
                case let .emailOTPVerify(viewModel):
                    OTPVerificationScreen(viewModel: viewModel)
                case let .emailSignUp(email):
                    EmailSignUpScreen(email: email)
                case let .emailPasswordLogin(email, authUser):
                    EmailPasswordScreen(email: email, authUser: authUser)
                case .phoneSignUp:
                    AuthUserUpdateScreen()
                }
            }
        }
    }
}
```

Navigation Stack Wrapper

The `CustomNavigation` struct acts as a robust wrapper around SwiftUI's `NavigationStack`, consolidating the logic for handling our primary, secondary, and tertiary navigation flows. This approach centralises navigation setup, making it more manageable and consistent across the application.

```
enum NavigationRouterType {
    case primary, secondary, tertiary
}

public struct CustomNavigation<Content>: View where Content: View {
    @StateObject private var primary = PrimaryNavigation.router
    @StateObject private var secondary = SecondaryNavigation.router
    @StateObject private var tertiary = TertiaryNavigation.router
    public let content: Content
    let currentPath: NavigationRouterType
    init(_ path: NavigationRouterType, @ViewBuilder content: @escaping () -> Content) {
        self.content = content()
        currentPath = path
    }

    public var body: some View {
        switch currentPath {
        case .primary:
            NavigationStack(path: $primary.path) {
                content
                .primaryNavigationDestinations()
            }
            .onDisappear {
                primary.dismissAll()
            }
        case .secondary:
            NavigationStack(path: $secondary.path) {
                content
                .secondaryNavigationDestinations()
            }
            .onDisappear {
                secondary.dismissAll()
            }
        case .tertiary:
            NavigationStack(path: $tertiary.path) {
                content
                .tertiaryNavigationDestinations()
            }
            .onDisappear {
                tertiary.dismissAll()
            }
        }
    }
}
```

Set up in Your App

```
struct MainTabScreen: View {
  @StateObject var viewModel = MainTabViewModel()

  var body: some View {
    CustomNavigation(.primary) {
      TabView(selection: $viewModel.selection) {
        HomeScreen()
          .tag(MainTabItem.home)\
          .tabItem {
            TabBarItem(selected: viewModel.selection, item: .home)
          }
        WishListScreen()
          .tag(MainTabItem.wishlist)
          .tabItem {
            TabBarItem(selected: viewModel.selection, item: .wishlist)
          }
        TripHistoryScreen()
          .tag(MainTabItem.trip)
          .tabItem {
            TabBarItem(selected: viewModel.selection, item: .trip)
          }
        ProfileScreen()
          .tag(MainTabItem.profile)
          .tabItem {
            TabBarItem(selected: viewModel.selection, item: .profile)
          }
      }
    }
  }
}
```

Usage

```
// In a view or view model
Button("View Details") {
    PrimaryNavigation.router.present(
        .hotel(.details(hotelInput))
    )
}

func retrievePaymentStatus(_ notification: NotificationCenter.Publisher.Output) {
    if status == .success {
        // Clear navigation stack after payment success
        PrimaryNavigation.router.dismissAll()
    }
}
```

Async + Navigation Flows

```
func loginPhone() async {  
    isLoading = true  
    defer { isLoading = false }  
  
    let success = await sendVerification()  
    if success {  
        // Navigate on success  
        TertiaryNavigation.router.present(  
            .login(.phoneOTPVerify(self))  
        )  
    }  
}
```

Deep Linking Ready

Easy to add deep linking with this pattern:

```
func handleDeepLink(_ url: URL) {  
    let path = parsePath(url)  
    PrimaryNavigation.router.present(path)  
}
```

Key Advantages

Let's recap why this pattern is powerful:



Type Safety

Eliminates accidental navigation to invalid states through strong enum-based enforcement.



Testability

Routers are simple classes, enabling easy unit testing without the full SwiftUI view hierarchy.



Scalability

Adding new routes is simple: an enum case and a switch case in the destination modifier is all it takes.



Maintainability

Centralised routing logic means changes are easy to locate and implement efficiently.

Best Practices

As you implement this in your own applications, keep these practices in mind to ensure a robust and maintainable navigation system.



Group Routes by Feature

Avoid creating a single, massive enum. Instead, logically organise your routes by feature, such as `HotelPath`, `FlightPath`, and `AuthPath`. This enhances modularity and clarity.



Descriptive Enum Cases

Ensure your enum cases are highly descriptive. For example, `case hotelDetails(HotelInputModel)` is far more informative than a generic `case detail(HotelInputModel)`, making your code easier to understand.



Isolate Router Testing

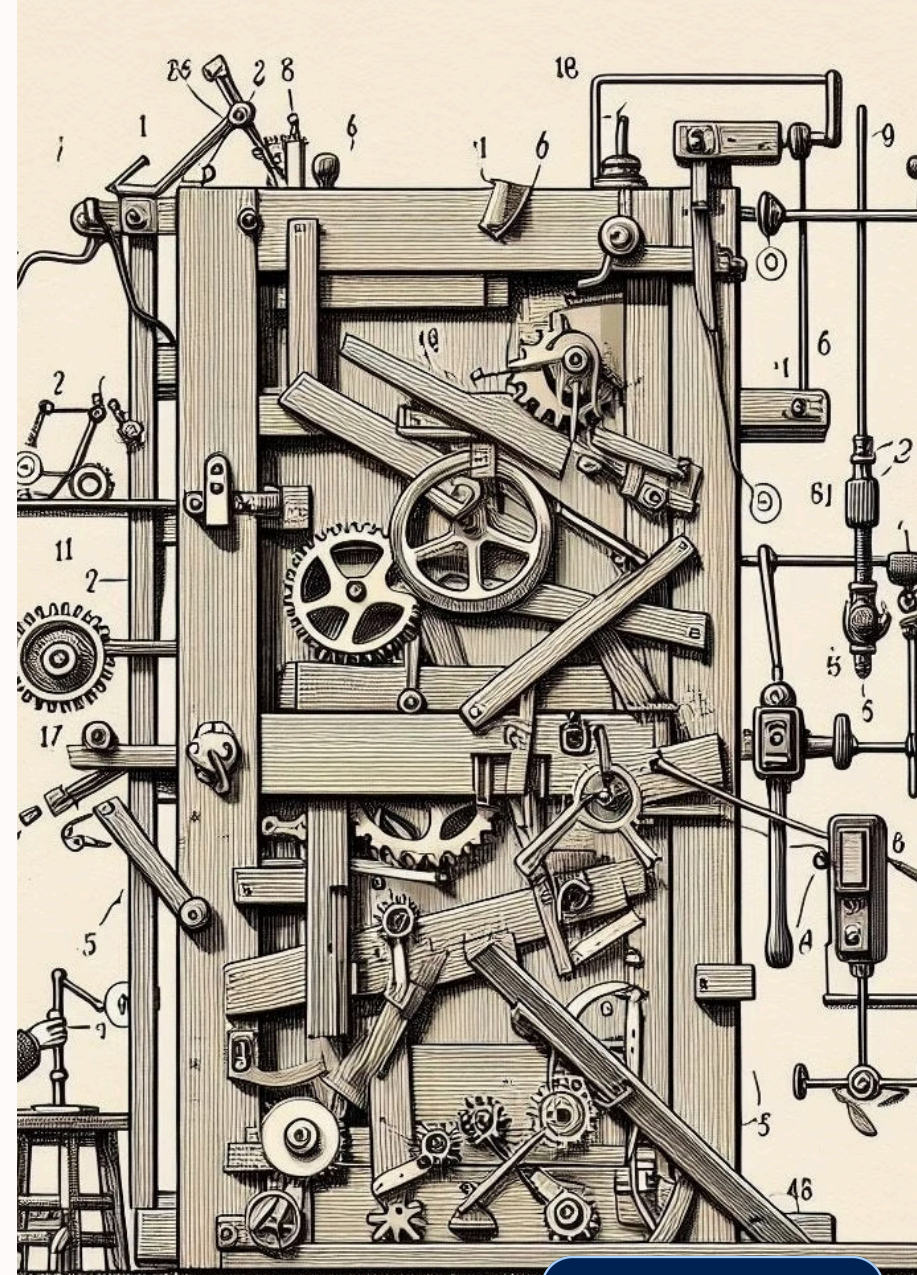
Treat routers as pure classes that can be tested in isolation. Verify that `present` correctly adds to the path and `dismiss` removes it, without coupling your tests to the full SwiftUI view hierarchy.

Common Pitfalls

“

Over-engineering small apps: If you're building a single-screen app, this pattern is overkill. Use it when you have multiple connected flows.

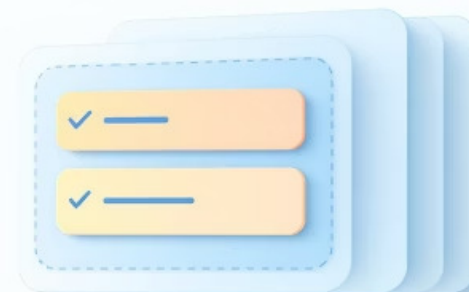
”



Testing Routers

Let's examine how this architecture simplifies the process of testing your navigation logic.

```
func testNavigation() async {  
    let router = PrimaryNavigation.router  
    router.present(.hotel(.listing(input)))  
  
    XCTAssertEqual(router.path.count, 1)  
  
    router.dismiss()  
    XCTAssertEqual(router.path.count, 0)  
}
```



Key Takeaways

This architectural pattern delivers several compelling advantages for building robust and scalable SwiftUI applications:



Type Safety & Testability

Enum-based routing ensures compile-time checks, eliminating invalid navigation states. Routers are pure, testable classes, decoupled from the UI.



Singleton Efficiency

The Singleton pattern provides easy access to router instances throughout your app, managing navigation state centrally and predictably.



Seamless Async Integration

Naturally supports asynchronous operations, allowing navigation to occur reliably after network requests or other background tasks complete.



Scalable & Adaptable

Designed to grow with your application, from simple flows to complex enterprise-level navigation structures with multiple concurrent stacks.

Thank you Thank You

I hope this presentation has provided valuable insights into architecting robust and scalable navigation for your real-world applications.

Please feel free to connect with me for further discussion:

<https://www.linkedin.com/in/sukritha/>